SyncSpace: Project Overview

Welcome to **SyncSpace**! This document is designed to help you, as a beginner developer, understand exactly what this project is, how it works, and what tools are used to build it.

🌟 What is SyncSpace?

SyncSpace is a **production-grade real-time collaborative code editor**. Think of it like Google Docs, but for writing code! Multiple users can join a "room," write code together simultaneously without overwriting each other's work, and then execute (run) that code to see the results.

To make this secure and efficient, the project is divided into three separate parts (or "microservices"):

1. **Client (Frontend)**: The user interface that you see in your browser.
2. **Server (Backend)**: The central brain that handles user accounts, saves data, and broadcasts real-time code changes to all users.
3. **Execution Engine**: A separate, secure environment whose sole job is to safely run the code submitted by users.

Here is a detailed breakdown of each part.

💻 1. The Client (Frontend)

Location: `/client` folder

The frontend is what the user interacts with. It contains the code editor, the file explorer, the login screens, and the dashboard.

Folder Structure

```
client/
├── src/
│   ├── assets/       # Static assets like images and global styles
```

```
    ├── components/   # Reusable UI parts (e.g., EditorWorkspace, FileExplorer, TerminalOutput, UserSidebar)
    ├── pages/        # Main application views tied to routes (Dashboard, Workspace, Login, Register)
    ├── services/     # Logic for making HTTP API calls to the Backend
    ├── store/        # Zustand global state management files
    ├── App.tsx       # Main React Router component routing the pages
    └── main.tsx      # The application's main entry point
```

Workflow & Interaction

- **With the Server (HTTP):** When a user logs in, creates a room, or loads their dashboard, the frontend components (via `/services`) use **Axios** to send HTTP requests to the Backend.
- **With the Server (WebSockets):** When a user opens a `Workspace`, the client establishes a persistent WebSocket connection to the Backend. The `EditorWorkspace` component connects the Monaco Editor to `yjs`, syncing keystrokes instantly to everyone else in the room.
- **With the Execution Engine:** When the user clicks the "Run" button in the workspace, the frontend extracts the code from the editor and sends an HTTP POST request directly to the Execution Engine. It then displays the result in the `TerminalOutput` component.

Key Packages & Libraries Used:

- **`react` & `react-dom`**: The core framework used to build the user interface using reusable components.
- **`vite`**: The build tool. It is incredibly fast at starting the development server and bundling the code for production.
- **`react-router-dom`**: Manages navigation between different pages.
- **`@monaco-editor/react`**: This is the core code editor component (the exact same editor that powers VS Code).
- **`yjs`**: The magic behind real-time collaboration. It is a "CRDT" library that merges changes made by multiple people at the exact same time.
- **`y-websocket`**: Connects `yjs` to our backend server over WebSockets.
- **`y-monaco`**: The bridge that connects our Monaco Editor to `yjs`.
- **`zustand`**: A small, fast state management library acting as a global "store" for variables.
- **`tailwindcss` & `@tailwindcss/postcss`**: A utility-first CSS framework for beautiful styling.
- **`lucide-react`**: A library providing beautiful SVG icons.
- **`axios`**: Used to make HTTP requests to our backend server.

⚙️ 2. The Server (Backend)

Location: `/server` folder

The backend is an API that manages the database, handles user authentication, manages the “rooms,” and acts as the central hub for the real-time WebSocket connections.

Folder Structure

```
server/
├── src/
│   ├── config/       # Database (MongoDB) connection logic
│   ├── controllers/  # The core logic for handling API requests (Auth, Rooms)
│   ├── middleware/   # Express middlewares (e.g., verifying JWT tokens, handling CORS)
│   ├── models/       # Mongoose schemas defining how data (User, Room) looks in the database
│   ├── routes/       # Express route definitions mapping URLs to specific controllers
│   ├── socket/       # WebSockets & Yjs synchronization logic (setupWebSocket)
│   └── index.js      # Main Express server entry point that ties everything together
```

Workflow & Interaction

- **With the Client (HTTP):** The `index.js` server listens for API requests. It routes these to `/routes`, which invoke `/controllers`. The controllers talk to MongoDB (via `/models`) to fetch or save data and return a JSON response back to the Client.
- **With the Client (WebSockets):** The `socket/` folder handles WebSocket upgrades. It maintains a live, in-memory representation of every active code document. When Client A types a letter, the Server receives it via WebSockets and immediately broadcasts it to Client B and Client C in the same room.
- **With Redis:** The Server talks to Redis to store a blacklist of logged-out tokens, ensuring secure session termination.

Key Packages & Libraries Used:

- **`express`**: A minimalist web framework for Node.js used to create our API routes.
- **`mongoose`**: An Object Data Modeling (ODM) library for MongoDB.
- **`ws`**: A fast WebSocket library for Node.js.
- **`y-websocket` & `yjs`**: Used to understand, merge, and broadcast real-time document updates.
- **`jsonwebtoken` (JWT)**: Used for secure authentication and generating login tokens.
- **`bcrypt`**: A security library used to securely hash passwords before storing them.
- **`ioredis`**: A robust client for Redis (used for the token blacklist).
- **`cors`**: A security middleware dictating which frontend websites are allowed to talk to our API.
- **`dotenv`**: Loads secret variables from a hidden `.env` file.

🚀 3. The Execution Engine

Location: `/execution-engine` folder

The Execution Engine acts as an isolated "sandbox". If a user writes malicious code (like an infinite loop or a command to delete system files), we don't want it crashing our main Server. Thus, code execution is completely decoupled from the Backend.

Folder Structure

```
execution-engine/
├── src/
│   ├── runner.js     # Core logic handling child_process execution, temp files, and timeouts
│   └── index.js      # A small Express server listening for the /execute endpoint
├── temp/             # A temporary directory where user code is briefly written and executed
├── Dockerfile        # Configuration for deploying the engine itself
├── Dockerfile.node   # Environment configuration for the Node.js sandbox
└── Dockerfile.python # Environment configuration for the Python sandbox
```

Workflow & Interaction

- **With the Client:** Operates entirely independently of the main Server. It simply waits for an HTTP POST request on `/execute` directly from the Client containing the user's source code and language (e.g., JavaScript or Python).
- **Internal Workflow (`runner.js`):**
  1. It generates a unique random session ID.
  2. It creates a folder in `/temp/` and writes the user's code to a file (like `script.js`).
  3. It uses Node's built-in `child_process` to execute the file exactly as a human would in a terminal.
  4. It enforces a strict timeout limit (e.g., 10 seconds) to kill infinite loops.
  5. It captures all terminal output (`stdout`) and errors (`stderr`), deletes the temporary folder, and sends the output back to the Client.

Key Packages & Libraries Used:

- **`express`**: Used to create a simple HTTP server that listens for incoming code execution requests.
- **`child_process` (Node.js built-in)**: Specifically, the `exec` method. This allows Node.js to run terminal commands.

- **`fs/promises` & `path` (Node.js built-in)**: Used to interact with the file system (creating/deleting temporary execution folders).
- **`cors` & `dotenv`**: Used to handle cross-origin requests and load environment variables respectively.

🌐 The Complete Big Picture (Step-by-Step)

1. **Authentication:** The `Client` sends login credentials to the `Server`. The `Server` checks MongoDB, generates a JWT, and sends it back.
2. **Collaboration:** The user enters a `Workspace`. The `Client` opens a persistent WebSocket connection to the `Server`.
3. **Real-Time Sync:** As the user types in the Monaco Editor, `yjs` captures the keystrokes. The `Client` sends these keystrokes instantly to the `Server` via WebSockets. The `Server` broadcasts these changes to everyone else in the room.
4. **Execution:** The user clicks "Run". The `Client` skips the main Server entirely, taking the raw code and sending an HTTP POST request to the `Execution Engine`. The engine writes the code to a file, safely runs it in the `/temp` folder, and returns the console output for the `Client` to display.

🏁 Summary

SyncSpace is a beautifully orchestrated trio of applications. By separating the user interface (Client), the data management (Server), and the risky code execution (Execution Engine), it remains fast, scalable, and secure. Using modern tools like Vite, Tailwind, Yjs, and Express makes it an excellent example of modern web development architecture!